Exercícios LAB-4

ENUNCIADO – Faça uma classe java contendo dois vetores ou use crie um objeto java para armazenar uma lista de nomes, e-mails e RGs; Se quiser você pode criar vetores com valores ao invés de fazer a leitura pelo teclado;
Todos os vetores devem ter a mesma quantidade de elementos (o nome completo de uma pessoa, email e rg).

O programa será comandado por um menu com com as seguintes opções:

```
1 Ler Nome, email e RG;
2 Exibir tudo lado a lado;
3 Classificar por nome usando Selection Sort,
4 classificar por nome usando Insert Sort,
5 Classificar por RG usando Selection Sort,
6 Classificar por RG usando Insert Sort,
7 Classificar por RG usando Merge Sort,
8 Classificar por e-mail usando Quick Sort,
9 Buscar um RG usando busca binária,
10 Buscar um nome usando busca binária.
0 Finalizar programa.
```

CÓDIGO:

Usuario.java (classe de dados para o nome e senha)

```java
public class Usuario {
   private String nome;
   private String rg;
   private String email;

   Usuario(String nome, String rg, String email) {
      this.nome = nome;
      this.rg = rg;
      this.email = email;
   }


   public String getNome() {
      return nome;
   }
   public String getRg() {
      return rg;
   }
   public String getEmail() { return email;}
   public void setNome(String nome) {
      this.nome = nome;
   }
   public void setRg(String rg) {
      this.rg = rg;
   }
   public void setEmail(String email) { this.email = email;}
}
```

Principal.java

```
import javax.swing.*;
import java.util.Arrays;
import java.util.function.Function;

public class Principal {

    public static final int MAX = 5;
    static final Usuario[] usuarios = new Usuario[MAX];
    static int ultimaPosicao = -1;

    public static void main(String[] args) {
        carregarDados();

        String  txtMenu =  "1.  Ler nome, e-mail e RG\n";
                txtMenu += "2.  Listar nomes, e-mail e RG\n";
                txtMenu += "3.  Classificar por nome (bubblesort) \n";
                txtMenu += "4.  Classificar por nome (selection sort)\n";
                txtMenu += "5.  Classificar por RG (selection sort)\n";
                txtMenu += "6.  Classificar por RG (insertion sort)\n";
                txtMenu += "7.  Classificar por RG (merge sort)\n";
                txtMenu += "8.  Classificar por e-mail (quick sort)\n";
                txtMenu += "9.  Buscar RG (binary search)\n";
                txtMenu += "10. Buscar nome (binary search)\n";
                txtMenu += "0.  Sair \n\n\n";

        boolean executando = true;

        do{
            int opcao = Integer.parseInt(JOptionPane.showInputDialog(null, txtMenu,
"MENU INICIAL", JOptionPane.PLAIN_MESSAGE));

            switch (opcao) {
                case 1 -> cadastrar();
                case 2 -> listar(usuarios);
                case 3 -> listarNomeBubbleSort();
                case 4 -> listarNomeSelectionSort();
                case 5 -> listarRGSelectionSort();
                case 6 -> listarRGInsertionSort();
                case 7 -> listarRGMergeSort();
                case 8 -> listarEmailQuickSort();
                case 9 -> buscarRG();
                case 10 -> buscarNome();
                case 0 -> executando = false;
            }
        }while(executando);

        JOptionPane.showMessageDialog(null, "Sistema encerrado");
        System.exit(0);
    }

    public static void cadastrar() {
        if(ultimaPosicao == usuarios.length - 1) {
```

```
            JOptionPane.showMessageDialog(null, "Cadastros já realizados!");
            return;
        }

        int opcao;

        do {
            int i = ultimaPosicao + 1;

            String nome = JOptionPane.showInputDialog("Digite o nome do " + (i+1) +
"º usuário: ").toLowerCase();
            nome = nome.substring(0, 1).toUpperCase() + nome.substring(1);
//capitalize name

            String rg = JOptionPane.showInputDialog("Digite a rg do " + (i+1) + "º
usuário: ");

            String email = JOptionPane.showInputDialog("Digite o e-mail do " + (i+1) +
"º usuário: ").toLowerCase();

            Usuario usuario = new Usuario(nome, rg, email);

            usuarios[i] = usuario;
            ultimaPosicao = i;

            if(ultimaPosicao < usuarios.length - 1) {
                String resposta = JOptionPane.showInputDialog("Deseja cadastrar outro
usuário? [S/N] ");
                opcao = resposta.equalsIgnoreCase("S") ? 1 : 0;
            }else {
                opcao = 0;
            }

        }while(opcao == 1);

    }

    public static void listar(Usuario[] listaUsuarios) {
        String saida = "\n\n";

        //enhanced for
        for(Usuario usuario : listaUsuarios) {
            if(usuario != null) {
                saida += "Nome: %s | E-mail: %s | RG: %s\
n".formatted(usuario.getNome(), usuario.getEmail(), usuario.getRg());
            }
        }

        JOptionPane.showMessageDialog(null, saida);
    }

    public static void listarNomeBubbleSort() {
        Usuario[] lista = bubbleSort(usuarios, Usuario::getNome);
```

```
        listar(lista);
    }

    public static void listarNomeSelectionSort() {
        Usuario[] lista = selectionSort(usuarios, Usuario::getNome);

        listar(lista);
    }

    public static void listarRGSelectionSort() {
        Usuario[] lista = selectionSort(usuarios, Usuario::getRg);

        listar(lista);
    }

    public static void listarRGInsertionSort() {
        Usuario[] lista = insertionSort(usuarios, Usuario::getRg);

        listar(lista);
    }

    public static void listarRGMergeSort() {
        Usuario[] lista = usuarios.clone();
        mergeSort(lista);

        listar(lista);
    }

    public static void listarEmailQuickSort() {
        Usuario[] lista = usuarios.clone();
        quickSort(lista, 0, lista.length - 1);

        listar(lista);
    }


    private static void buscarRG() {
        String rg = JOptionPane.showInputDialog("Digite o RG desejado: ");

        Usuario[] usuariosOrdenados = bubbleSort(usuarios, Usuario::getRg);

        int pos = buscaBinaria(rg, Usuario::getRg, usuariosOrdenados);

        if(pos == -1) {
            JOptionPane.showMessageDialog(null, "Usuário com o RG " + rg + " não
encontrado!");
        }else {
            Usuario u = usuariosOrdenados[pos];
            JOptionPane.showMessageDialog(null, "Nome: %s | E-mail: %s | RG: %s\
n".formatted(u.getNome(), u.getEmail(), u.getRg()));
        }
    }
```

```java
    private static void buscarNome() {
        String nome = JOptionPane.showInputDialog("Digite o nome desejado: ");

        Usuario[] usuariosOrdenados = bubbleSort(usuarios, Usuario::getNome);

        int pos = buscaBinaria(nome, Usuario::getNome, usuariosOrdenados);

        if(pos == -1) {
            JOptionPane.showMessageDialog(null, "Usuário com o nome " + nome + "
não encontrado!");
        }else {
            Usuario u = usuariosOrdenados[pos];
            JOptionPane.showMessageDialog(null, "Nome: %s | E-mail: %s | RG: %s\
n".formatted(u.getNome(), u.getEmail(), u.getRg()));
        }
    }



    /**
     * Realiza uma busca em um vetor dado para encontrar o valor informado
utilizando o algoritmo de Busca binária.
     * NOTA: o array informado deve estar previamente informado, caso contrário o
comportamento da função é indefinido.
     * @param busca o valor que se deseja encontrar.
     * @param attrExtractor interface funcional para buscar o atributo do tipo
String que se deseja usar do objeto.
     * @param vetorOrdenado vetor previamente ordenado da onde se deseja
buscar o valor informado.
     * @return índice em que o item se encontra no array informado ou -1 se não
for encontrado.
     */
    private static int buscaBinaria(String busca, Function<Usuario, String>
attrExtractor, Usuario[] vetorOrdenado) {
        int pInicio = 0;
        int pFinal = ultimaPosicao;
        int pMeio = (pInicio + pFinal) / 2;

        while(pInicio <= pFinal) {
            if(busca.equalsIgnoreCase(attrExtractor.apply(vetorOrdenado[pMeio]))) {
                return pMeio;
            }else
if( busca.compareToIgnoreCase(attrExtractor.apply(vetorOrdenado[pMeio])) > 0) {
                pInicio = pMeio + 1; //descarta o meio
            }else {
                pFinal = pMeio - 1; //descarta o meio
            }

            pMeio = (pInicio + pFinal) / 2;
        }

        return -1;
    }
```

```
   private static Usuario[] bubbleSort(Usuario[] vetor, Function<Usuario, String>
attrExtractor) {
      Usuario[] mVetor = vetor.clone();

      boolean troca = true;
      while(troca) {
         troca = false;
         for(int i = 0;i < ultimaPosicao; i++) {
            if(ehMaior(attrExtractor.apply(mVetor[i]),
attrExtractor.apply(mVetor[i+1]))) {
               trocar(mVetor, i, i+1);
               troca = true;
            }
         }
      }

      return mVetor;
   }

   private static Usuario[] selectionSort(Usuario[] vetor, Function<Usuario, String>
attrExtractor) {
      Usuario[] mVetor = vetor.clone();

      int posUlt = ultimaPosicao;

      while(posUlt > 0) {
         int posMaior = 0;

         for(int i = 0; i < posUlt; i++) {
            posMaior = ehMaior(attrExtractor.apply(mVetor[posMaior]),
attrExtractor.apply(mVetor[i+1])) ? posMaior : i+1;
         }
         trocar(mVetor, posMaior, posUlt);
         posUlt--;
      }

      return mVetor;
   }

   private static Usuario[] insertionSort(Usuario[] vetor, Function<Usuario, String>
attrExtractor) {
      Usuario[] mVetor = vetor.clone();

      int posUlt = ultimaPosicao;

      while(posUlt > 0) {
         for(int i = 0; i < posUlt; i++) {
            if(ehMaior(attrExtractor.apply(mVetor[i]),
attrExtractor.apply(mVetor[posUlt]))) {
               trocar(mVetor, i, posUlt);
            }
         }

         posUlt--;
```

```
        }

        return mVetor;
    }

    //devolve 'true' apenas se str1 é maior
    private static boolean ehMaior(String str1, String str2) {
        for(int i = 0; i < str1.length(); i++) {
            char c1 = str1.charAt(i);
            char c2;

            try {
                c2 = str2.charAt(i);
            }catch (IndexOutOfBoundsException e) { //a str2 é menor. ex: str1 = "abc"
e str2 = "ab"
                return true;
            }

            if(c1 > c2) {
                return true;
            } else if(c2 > c1) {
                return false;
            }
        }

        return false; //as strings são iguais
    }

    private static void trocar(Usuario[] vetor, int i1, int i2) {
        Usuario tmp = vetor[i1];
        vetor[i1] = vetor[i2];
        vetor[i2] = tmp;
    }


    public static void carregarDados() {
        Usuario[] users = {
                new Usuario("Allan", "111", "allan@fds.com"),
                new Usuario("Alex", "222", "alex@xxx.com"),
                new Usuario("Yasmin", "888", "yasmin@bebe.love"),
                new Usuario("Ellen", "777", "lomy@elephant.com.br"),
                new Usuario("Teté", "0", "tt2020@fds.com"),
        };

        //copia um array a partir de um índice para outro a partir de um índice, e o
tamanho desejado
        System.arraycopy(users, 0, usuarios, 0, usuarios.length);

        ultimaPosicao = 4;
    }


    private static void mergeSort(Usuario[] array) {
        if(array.length > 1) {
```

```
        //quebrar em dois
        int mid = array.length / 2;

        Usuario[] leftArray = Arrays.copyOfRange(array, 0, mid);
        Usuario[] rightArray = Arrays.copyOfRange(array, mid, array.length);

        //aplicar o mergesort em cada metade
        mergeSort(leftArray);
        mergeSort(rightArray);

        //juntas as metades
        merge(leftArray, rightArray, array);
    }
  }

  //modifica o array arr
  private static void merge(Usuario[] left, Usuario[] right, Usuario[] arr) {
    int l = 0; //indice vetor esquerdo
    int r = 0; //indice vetor direito
    int i = 0; //indice vetor principal

    while(l < left.length || r < right.length) {
      if(l >= left.length) { //lado esquerdo já mesclado
        arr[i] = right[r++];
      }else if(r >= right.length) { //lado direito já mesclado
        arr[i] = left[l++];
      }else {
        if(left[l].getRg().compareToIgnoreCase(right[r].getRg()) <= 0) {
          arr[i] = left[l++];
        }else {
          arr[i] = right[r++];
        }
      }
      i++;
    }
  }

  private static void quickSort(Usuario[] array, int inicio, int fim) {
    //executa apenas se tiver 2 ou mais elementos
    if(fim - inicio > 0) {
      //separa o array em menores que o pivô e maiores que o pivô e retorna a
posição final do pivô na partição
      int iPivot = partition(array, inicio, fim);

      //chama recursivamente para cada metade
      quickSort(array, inicio, iPivot - 1);
      quickSort(array, iPivot + 1, fim);
    }
  }

  private static int partition(Usuario[] array, int inicio, int fim) {

    //faz uma pré-ordenação e devolve o pivô na posição fim - 1 (penúltima do
array)
```

```
        int iPivot = choosePivot(array, inicio, fim);

        //índice que guarda os números menores/iguais que o pivô (já considera a posição inicio, por causa da pré-ordenação
        int i = inicio-1;

        //percorre o array a partir da 2ª posição e até a penúltima (devido à pré-ordenação do choosePivot)
        for(int j = inicio; j < fim - 1; j++) {
            //realiza troca se for menor/igual que o pivô
            if(array[j].getEmail().compareToIgnoreCase(array[iPivot].getEmail()) <= 0) {
                ++i;
                Usuario swap = array[i];
                array[i] = array[j];
                array[j] = swap;
            }
        }

        //coloca o pivot na posição correta
        ++i;
        Usuario swap = array[i];
        array[i] = array[iPivot];
        array[iPivot] = swap;

        return i;
    }

    //escolhe a mediana de 3 entre o primeiro elemento, o último elemento e o elemento central como pivô
    private static int choosePivot(Usuario[] array, int inicio, int fim) {
        int mid = inicio + (fim - inicio) / 2;

        if(array[inicio].getEmail().compareToIgnoreCase(array[mid].getEmail()) > 0) {
            Usuario swap = array[inicio];
            array[inicio] = array[mid];
            array[mid] = swap;
        }
        if(array[inicio].getEmail().compareToIgnoreCase(array[fim].getEmail()) > 0) {
            Usuario swap = array[inicio];
            array[inicio] = array[fim];
            array[fim] = swap;
        }
        if(array[mid].getEmail().compareToIgnoreCase(array[fim].getEmail()) > 0) {
            Usuario swap = array[mid];
            array[mid] = array[fim];
            array[fim] = swap;
        }

        //joga o pivot para a penúltima posição do array
        Usuario swap = array[mid];
        array[mid] = array[fim - 1];
        array[fim - 1] = swap;
```

```
        return fim - 1;
    }
}
```

PRINTS DE EXECUÇÃO:

